

02. Đo Chất lượng không khí ESP32 20224301 (2)

Kỹ thuật vi xử lý (Trường Đại học Bách khoa Hà Nội)



messages.pdf_cover_qr_code_label

HA NOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING



Project Report

Thiết bị đo chất lượng không khí

Class: *ET3302E – 152445*

Lecturer: *Dr.Han Huy Dung*

Group: *2*

Student: *Bùi Văn Bình – 20224301*

Vũ Minh Đức - 20224305

Trịnh Quốc An - 20224269

Hanoi, 20/10/2024

PREFACE

In recent years, air quality has become an increasingly urgent issue, especially in urban areas and developed industrial parks. Air pollution not only seriously affects human health, but also harms the natural environment and ecosystems. In Vietnam, a country in the process of industrialization, monitoring and controlling air quality has become more necessary than ever to ensure sustainable development.

In the course of studying Microprocessor Engineering, we realized that the knowledge learned can be applied to develop air quality monitoring devices, thereby contributing to the protection of public health. Therefore, we chose the topic "Air Quality Measuring Equipment" to realize this idea. The topic is not only an opportunity to apply theory from the lecture hall but also an opportunity to practice and consolidate professional knowledge.

We would like to sincerely thank MSc. Han Huy Dung, who has dedicated his guidance and support throughout the implementation of this project.

MỤC LỤC

Không tìm thấy mục nào của mục lục.

1. GIỚI THIỆU

1.1 Vấn đề

Hiện nay, chất lượng không khí tại Việt Nam, đặc biệt là tại Hà Nội, đang ở mức báo động nghiêm trọng. Theo số liệu từ Trung tâm Quan trắc môi trường (thuộc Tổng cục Môi trường), vào thời điểm cao điểm, chỉ số chất lượng không khí (AQI) của Hà Nội thường vượt ngưỡng 150, thậm chí có lúc đạt mức trên 200, đây là mức "rất có hại cho sức khỏe". Trong đó, nồng độ bụi mịn PM2.5, tác nhân gây ô nhiễm chính, đã nhiều lần vượt quá giới hạn an toàn do Tổ chức Y tế Thế giới (WHO) đặt ra. Cuối năm 2023, nhiều khu vực tại Hà Nội ghi nhận chỉ số PM2.5 trung bình hàng năm cao gấp 4-5 lần ngưỡng khuyến cáo của WHO

Mới đây, sự kiện "Ô nhiễm kỷ lục Hà Nội" vào tháng 9/2023, khi chỉ số AQI ở một số khu vực của trung tâm thành phố đạt trên 300, mức "nguy hiểm" đã làm dấy lên những lo ngại lớn về sức khỏe cộng đồng

Ngoài vấn đề chất lượng không khí, Hà Nội cũng bị ảnh hưởng nặng nề bởi biến đổi khí hậu, nhiệt độ và độ ẩm thay đổi thất thường. Các đợt nắng nóng khắc nghiệt vào mùa hè đã khiến nhiệt độ ban ngày thường xuyên vượt quá 40°C, trong khi vào mùa đông, nhiệt độ giảm đột ngột kết hợp với độ ẩm thấp tạo điều kiện thuận lợi cho các bệnh về đường hô hấp phát triển. Đặc biệt, những cơn mưa sớm kèm theo khói bụi và ô nhiễm không khí đã dẫn đến sự gia tăng các bệnh như viêm phổi, hen suyễn và các vấn đề về mắt

1.2 Ý tưởng và mục tiêu

Xuất phát từ nhu cầu cấp thiết phải theo dõi và cải thiện chất lượng không khí, nhóm nghiên cứu đã thiết kế một sản phẩm đột phá để thu thập dữ liệu môi trường như PM2.5, PM10, nồng độ nhiệt độ và độ ẩm. Thông qua việc phân tích các thông số này, thiết bị có thể cung cấp thông tin kịp thời về ô nhiễm không khí, giúp người dùng hiểu rõ hơn về môi trường xung quanh.

Thông tin chất lượng không khí sẽ được hiển thị thông qua giao diện trực quan, dễ sử dụng, giúp người dùng nắm bắt tình hình ngay lập tức và có biện pháp đối phó phù

hợp để bảo vệ sức khỏe của mình. Thiết bị cũng có khả năng cảnh báo khi các chỉ số ô nhiễm vượt quá ngưỡng an toàn, đảm bảo người dùng nhận được thông báo nhanh chóng và rõ ràng

Không chỉ thu thập và cảnh báo, nhóm còn chú trọng cải thiện hiệu suất của thiết bị bằng cách tối ưu hóa quy trình thu thập dữ liệu, cải tiến thuật toán phân tích, đảm bảo tính ổn định của hệ thống. Kết quả là một máy đo chất lượng không khí đáng tin cậy đóng góp quan trọng vào việc bảo vệ sức khỏe cộng đồng và nâng cao nhận thức về ô nhiễm không khí.

1.3 Phân tích nhu cầu

Trong bối cảnh hiện nay, ô nhiễm không khí là một vấn đề nghiêm trọng, đặc biệt là ở các thành phố lớn như Hà Nội, TP. Chỉ số chất lượng không khí (AQI) thường xuyên cao, ảnh hưởng xấu đến sức khỏe cộng đồng, đặc biệt là đối với những người mắc các bệnh về đường hô hấp, trẻ em và người già. Theo WHO, ô nhiễm không khí là nguyên nhân gây ra nhiều bệnh như hen suyễn, viêm phổi và thậm chí cả ung thư phổi. Điều này tạo ra nhu cầu cấp thiết để theo dõi và kiểm soát chất lượng không khí hàng ngày

Hiện nay, phần lớn người dân thiếu các công cụ giám sát chất lượng không khí tại nhà và nơi làm việc của họ, trong khi các thiết bị đo chất lượng không khí hiện có trên thị trường thường đắt tiền, khó tiếp cận hoặc không cung cấp dữ liệu tức thời và chi tiết. Điều này tạo ra nhu cầu về một thiết bị đo chất lượng không khí vừa đáng tin cậy, dễ sử dụng vừa giá cả phải chăng.

Hiện nay, phần lớn người dân thiếu các công cụ giám sát chất lượng không khí tại nhà và nơi làm việc của họ, trong khi các thiết bị đo chất lượng không khí hiện có trên thị trường thường đắt tiền, khó tiếp cận hoặc không cung cấp dữ liệu tức thời và chi tiết. Điều này tạo ra nhu cầu về một thiết bị đo chất lượng không khí vừa đáng tin cậy, dễ sử dụng vừa giá cả phải chăng.

2. CƠ SỞ LÝ THUYẾT

2.1 Giao thức HTTP

HTTP (Hypertext Transfer Protocol) là một giao thức truyền thông chuẩn được sử dụng rộng rãi trong việc trao đổi dữ liệu giữa máy tính và máy chủ trên World Wide Web (WWW). Được tạo ra bởi Tim Berners-Lee tại CERN vào những năm 1989-1991, HTTP đã trở thành nền tảng cơ bản cho việc truyền tải thông tin trên Internet.

Các đặc điểm quan trọng của giao thức HTTP bao gồm:

- **Giao thức Request-Response:** HTTP hoạt động dựa trên mô hình yêu cầu-đáp ứng. Máy tính gửi một yêu cầu HTTP tới máy chủ, yêu cầu này chứa các thông tin như phương thức (GET, POST, PUT, DELETE), URL và các tham số. Máy chủ xử lý yêu cầu và trả về một đáp ứng HTTP chứa dữ liệu được yêu cầu hoặc mã lỗi.
- **Phương Thức HTTP:** HTTP hỗ trợ một số phương thức chính, bao gồm:
 - GET: Lấy dữ liệu từ máy chủ.
 - POST: Gửi dữ liệu tới máy chủ để xử lý.
 - PUT: Cập nhật dữ liệu đã tồn tại trên máy chủ.
 - DELETE: Xóa dữ liệu trên máy chủ.
- **URL (Uniform Resource Locator):** Mỗi tài nguyên trên Internet được định danh bằng một URL, chứa thông tin về địa chỉ của tài nguyên và cách truy cập nó.
- **Stateless Protocol:** HTTP không duy trì trạng thái (stateless), điều này có nghĩa là mỗi yêu cầu được xử lý độc lập và không có thông tin về trạng thái trước đó.

- **Các Mã Trạng Thái (Status Codes):** HTTP sử dụng các mã trạng thái để thể hiện kết quả của mỗi yêu cầu, bao gồm 200 OK (thành công), 404 Not Found (không tìm thấy), 500 Internal Server Error (lỗi máy chủ), và nhiều mã khác.
- **Cơ Chế Bảo Mật:** HTTP hỗ trợ cơ chế bảo mật bằng cách sử dụng HTTPS (HTTP Secure), một phiên bản được mã hóa của giao thức HTTP, sử dụng SSL/TLS để bảo vệ dữ liệu truyền qua mạng.

HTTP đã trở thành một giao thức quan trọng cho việc trao đổi thông tin trên Internet và World Wide Web, đóng vai trò quan trọng trong việc cung cấp nội dung, dịch vụ và ứng dụng trực tuyến cho người dùng trên toàn cầu.

2.2 Giao thức giao tiếp WebSocket

WebSocket là kết nối liên tục giữa máy khách và máy chủ cho phép giao tiếp hai chiều giữa cả hai bên bằng kết nối TCP. Điều này có nghĩa là bạn có thể gửi dữ liệu từ máy khách đến máy chủ và từ máy chủ đến máy khách tại bất kỳ thời điểm nào.

Máy khách thiết lập kết nối WebSocket với máy chủ thông qua một quá trình được gọi là *bắt tay WebSocket*. Bắt tay bắt đầu bằng yêu cầu / phản hồi HTTP, cho phép máy chủ xử lý các kết nối HTTP cũng như kết nối WebSocket trên cùng một cổng. Sau khi kết nối được thiết lập, máy khách và máy chủ có thể gửi WebDữ liệu Socket ở chế độ song công.

- **HTTP truyền thống:**
 - Mỗi lần client cần dữ liệu, nó phải gửi một yêu cầu mới đến server (polling), gây ra độ trễ và tốn tài nguyên.
- **WebSocket:**
 - Sau khi thiết lập kết nối, client và server có thể gửi và nhận dữ liệu bất kỳ lúc nào, không cần tạo lại kết nối.

Sử dụng giao thức WebSockets, máy chủ (bo mạch ESP32) có thể gửi thông tin đến máy khách hoặc tất cả các máy khách mà không cần được yêu cầu. Điều này cũng cho phép chúng tôi gửi thông tin đến trình duyệt web khi có thay đổi xảy ra.

Dự án này cũng sẽ ứng dụng tính tiện dụng của Websocket :

- + **Cập nhật dữ liệu cảm biến theo thời gian thực:** Dữ liệu từ cảm biến (bụi, nhiệt độ, độ ẩm) có thể được truyền trực tiếp từ ESP32 đến trình duyệt mà không cần client phải gửi yêu cầu liên tục.
- + **Tiết kiệm băng thông:** So với HTTP polling, WebSocket tiết kiệm băng thông do không cần gửi các yêu cầu HTTP lặp lại.
- + **Cải thiện trải nghiệm người dùng:** Giao diện web sẽ hiển thị dữ liệu cảm biến gần như ngay lập tức khi có thay đổi.

Và để xây dựng máy chủ web, chúng ta sẽ sử dụng thư viện ESPAsyncWebServer. Thư viện này cần thư viện [AsyncTCP](#) để hoạt động bình thường.

2.3 Giao tiếp ADC

Giao tiếp với Analog-to-Digital Converter (ADC) là một khía cạnh quan trọng trong việc chuyển đổi dữ liệu analog thành dạng số, đặc biệt là trong các ứng dụng điện tử và hệ thống nhúng. ADC đóng vai trò quan trọng trong việc chuyển đổi tín hiệu từ các thiết bị cảm biến, cấp điện và các nguồn tín hiệu analog khác thành dữ liệu kỹ thuật số mà các vi điều khiển và vi xử lý có thể hiểu và xử lý.

Giao tiếp với ADC thường liên quan đến việc sử dụng các giao thức truyền thông như SPI (Serial Peripheral Interface) hoặc I2C (Inter-Integrated Circuit). Các thông số như độ nhạy, phân giải, và tốc độ lấy mẫu của ADC đóng vai trò quan trọng trong quá trình thiết kế hệ thống, đặc biệt là khi đòi hỏi sự chính xác và độ tin cậy cao trong việc đo đạc và xử lý tín hiệu analog.

Hiểu rõ về cách giao tiếp với ADC là quan trọng để tối ưu hóa chất lượng dữ liệu số và đảm bảo tính ổn định của hệ thống. Bằng cách này, họ có thể tận dụng tối đa khả năng của ADC và tích hợp hiệu quả vào các ứng dụng và sản phẩm của mình.

2.4 Giao tiếp I2C

Giao tiếp qua chuẩn I2C (Inter-Integrated Circuit) là một phương pháp hiệu quả và linh hoạt được sử dụng rộng rãi trong việc kết nối giữa các vi điều khiển và các thiết bị ngoại vi trong các hệ thống điện tử. I2C giúp đơn giản hóa quá trình truyền thông dữ liệu giữa các thành phần, đặc biệt là trong các ứng dụng nhúng và IoT.

Giao tiếp I2C sử dụng hai dây truyền thông chính: dây dữ liệu (SDA) và dây đồng hồ (SCL). Điều này cho phép nhiều thiết bị được kết nối trên cùng một bus, với mỗi thiết bị có một địa chỉ duy nhất. Quy trình truyền dữ liệu bao gồm các tín hiệu START và STOP, cùng với các khung dữ liệu chứa thông tin từ master đến slave hoặc ngược lại.

Giao tiếp I2C được ưa chuộng bởi sự linh hoạt và khả năng kết nối nhiều thiết bị trên cùng một bus, giảm thiểu việc sử dụng chân đầu ra trên vi điều khiển. Tốc độ truyền dữ liệu có thể linh hoạt điều chỉnh tùy thuộc vào yêu cầu của hệ thống. Việc hiểu rõ về giao tiếp I2C là quan trọng để tích hợp hiệu quả các thiết bị và mô-đun vào hệ thống của họ, đồng thời tối ưu hóa hiệu suất và khả năng mở rộng của hệ thống.

3. YÊU CẦU THIẾT KẾ

3.1 Yêu cầu chức năng

Với vai trò là một hệ thống đo chất lượng không khí, thiết bị của nhóm cần phải đáp ứng được các yêu cầu sau:

- Hệ thống phát hiện nhanh chóng, chính xác về các chỉ số chất lượng bụi trong không khí AQI : Thiết bị có cảm biến nhiệt độ, độ ẩm, khí liên quan đến chỉ số chất lượng môi trường hiện tại từ đó đánh giá chất lượng không khí tổng quan.
- Có hệ thống web server để hiển thị thông báo kết quả đo và đánh giá lên website
- Sử dụng thuật toán, từ đó đưa ra các mức, chất lượng không khí có an toàn từ đó đưa ra một số biện pháp.
- Mạch có jack DC để cấp nguồn và pin dự phòng.

3.2 Yêu cầu phi chức năng

Để đảm bảo hệ thống đo nhiệt độ, độ ẩm, và bụi hoạt động hiệu quả, tối ưu hóa trải nghiệm người dùng, cần đáp ứng các yêu cầu phi chức năng sau:

- **Thiết kế gọn nhẹ:** Sản phẩm phải có khả năng bố trí gọn gàng và linh hoạt, phù hợp để đặt tại các hộ gia đình, doanh nghiệp, hoặc các không gian khác.
- **Hoạt động ổn định:** Hệ thống cần đảm bảo đo lường liên tục và cập nhật chất lượng dữ liệu tốt nhất trong thời gian dài mà không xảy ra gián đoạn.
- **Tiết kiệm năng lượng:** Tổng công suất tiêu thụ không vượt quá 100W, giúp tiết kiệm điện năng.

- **Dải đo nhiệt độ:** Thiết bị cần đo được nhiệt độ trong khoảng từ **-10°C đến 170°C**.
- **Dải đo bụi (PM2.5):** Phạm vi đo chỉ số PM2.5 từ **400 ppm đến 10.000 ppm**, đáp ứng các tiêu chuẩn chất lượng không khí.
- **Tốc độ xử lý:** Thời gian xử lý và cập nhật dữ liệu từ cảm biến không vượt quá **10 giây**.
- **Tần suất cập nhật:** Dữ liệu phải được cập nhật tự động lên hệ thống mỗi **30 phút** để đảm bảo tính thời gian thực.
- **Giao diện hiển thị:** Giao diện web cần thiết kế rõ ràng, bố cục trực quan, đơn giản, và dễ sử dụng.
- **Ngôn ngữ hỗ trợ:** Giao diện người dùng phải sử dụng **ngôn ngữ tiếng Việt** để phù hợp với đối tượng người dùng trong nước.

4. THIẾT KẾ VÀ LỰA CHỌN PHƯƠNG ÁN PHÙ HỢP

4.1 Phân tích, lựa chọn linh kiện

1. Khối điều khiển: Module ESP32-WROOM-32:

ESP32 [4] là một vi điều khiển tích hợp mạnh mẽ được sử dụng phổ biến trong các ứng dụng IoT. Nó kết hợp Wi-Fi và Bluetooth, có tốc độ xử lý cao, khả năng tiết kiệm năng lượng, và hỗ trợ nhiều giao tiếp như UART, SPI, I2C. ESP32 có thể lập trình bằng Arduino IDE, MicroPython và ESP-IDF. Đây là một nền tảng linh hoạt và mạnh mẽ cho các dự án IoT. ESP32 là sản phẩm kế thừa từ vi điều khiển ESP8266.

Ta có thể lập trình bằng nhiều ngôn ngữ khác nhau như C/C++, Python, NodeJs, Lua, Ngoài ra, hiện tại Espressif Systems hỗ trợ chính thức SDK cho ESP32 với Arduino IDE, vì vậy ta có thể yên tâm xây dựng các ứng dụng với Arduino ESP32.

Thông số	Module ESP32-WROOM-32
Hình ảnh	

	
MCU	Xtensa® Dual-Core 32-bit LX6 600 DMIPS
Điện áp nguồn	5V DC
Đầu vào/ra điện áp	3.3V DC
Băng tần	80 - 240 MHz
Giao thức WiFi	802.11B/g/n/E/I (802.11N @ 2.4 GHz lên đến 150 Mbit/s)
Tần số WiFi	2.4 – 2.5 GHz
Bluetooth	V4.2 - BLE
Bộ nhớ	448 Kbyte ROM, 520 Kbyte SRAM, 6 Kbyte SRAM trên RTC và QSPI Hỗ trợ đèn flash / SRAM chip
Chip USB-Serial	CP2102
Anten không dây	PCB
Giá thành	200.000VNĐ

Bảng 1. Thông số của Module ESP32

2. Cảm biến nhiệt độ, độ ẩm

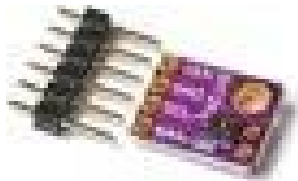
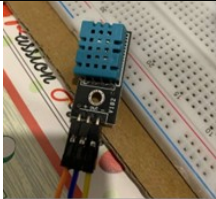
Cảm biến trong dự án này rất quan trọng, sẽ là thiết bị ngoại vi quan trọng và giúp dự án áp dụng thực tế và đưa ra được tính ứng dụng cao .

So sánh và chọn lựa loại cảm biến nhiệt độ độ ẩm phù hợp.

Nhóm chúng tôi chọn ra 3 loại chip cảm biến phù hợp với hệ thống cảm biến mức độ ổn cả về mặt hiệu suất, tính năng và thông số đo được :

Bảng dưới so sánh về các thông số cơ bản của các cảm biến :

Tên linh kiện	SHT31	DHT11	CJMCU-SHT10
---------------	--------------	--------------	--------------------

n			
Hình ảnh			
Độ nhạy	Thời gian phản hồi 2 đến 8s	Thời gian phản hồi trung bình 2s	Thời gian phản hồi từ 5 đến 30s
Dải đo	Nhiệt độ: - 40°C đến +125°C, Độ ẩm: 0% đến 100%	Nhiệt độ: - 0°C đến +50°C, Độ ẩm: 20% đến 80%	Nhiệt độ: - 40°C đến +123.8°C, Đ ộ ẩm: 5% đến 95%
Giao tiếp	I2C	1-Wire	Giao tiếp analog
Thông số đo được	Nhiệt độ, độ ẩm,	Nhiệt độ, độ ẩm	Nhiệt độ, độ ẩm
Độ sai số	Nhiệt độ: $\pm 0.2^{\circ}\text{C}$ Độ ẩm: $\pm 2\%$	Nhiệt độ: $\pm 0.2^{\circ}\text{C}$ Độ ẩm: $\pm 5\%$	Nhiệt độ: $\pm 0.5^{\circ}\text{C}$ Độ ẩm: $\pm 4\%$
Giá tiền	139.000đ	50,000đ	200.000đ

Bảng 5. So sánh thông số của một số loại cảm biến nhiệt độ, độ ẩm

È Sau qua trình so sánh , nhóm chúng tôi quyết định chọn cảm biến DHT11 vì nhận thấy rằng thiết bị này có dải đo lớn, giá thành rẻ, thời gian phản hồi nhanh và Cảm biến này có thể dễ dàng giao tiếp với bất kỳ bộ vi điều khiển để đo độ ẩm và nhiệt độ ngay lập tức.

3. Cảm biến bụi PM GP2Y1010AU0F

3.1. Giới thiệu

GP2Y1010AU0F Cảm Biến Bụi PM2.5 là một diode phát tia hồng ngoại (IRED) và một phototransistor được bố trí theo đường chéo vào thiết bị này. Nó phát hiện ánh sáng

phản xạ của bụi trong không khí. Đặc biệt, nó có hiệu quả phát hiện các hạt rất mịn như khói thuốc lá. Ngoài ra, nó có thể phân biệt khói với bụi nhà bằng dạng xung của điện áp đầu ra.

Mạch cảm biến này chủ yếu được sử dụng cho thiết bị báo động thiết bị loại bỏ bụi, thiết bị lọc không khí, rô bột bụi, báo cháy,...



- Nhận biết nồng độ bụi PM2.5 trong không khí

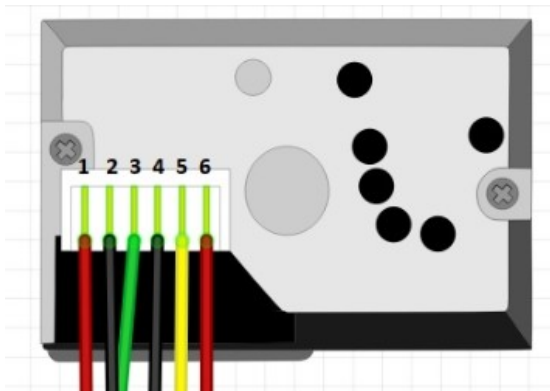
- Nguồn: 3.3 VDC

- Dòng tiêu thụ: 10mA

- Ngõ ra: analog với tỉ lệ $0.5V \sim 0.1mg/m^3$

- Nhiệt độ hoạt động: $-40 \sim 85$ độ C

Hình 1. Cảm biến bụi GP2Y1010AU0F SHARP



- 1 (V-LED): 3.3V Pin (150 Ohm in between)

- 2 (LED-GND): GND Pin

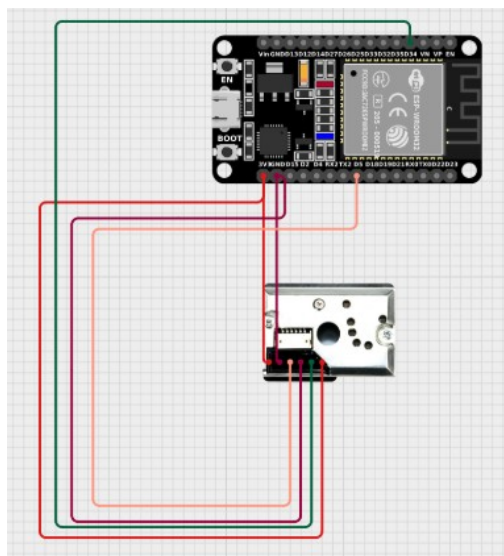
- 3 (LED): Digital Pin 12

- 4 (S-GND): GND Pin

- 5 (Vo): Analog Pin A6

- 6 (Vcc): 3.3V Pin (Direct)

Hình 2. Sơ đồ dây GP2Y1010AU0F



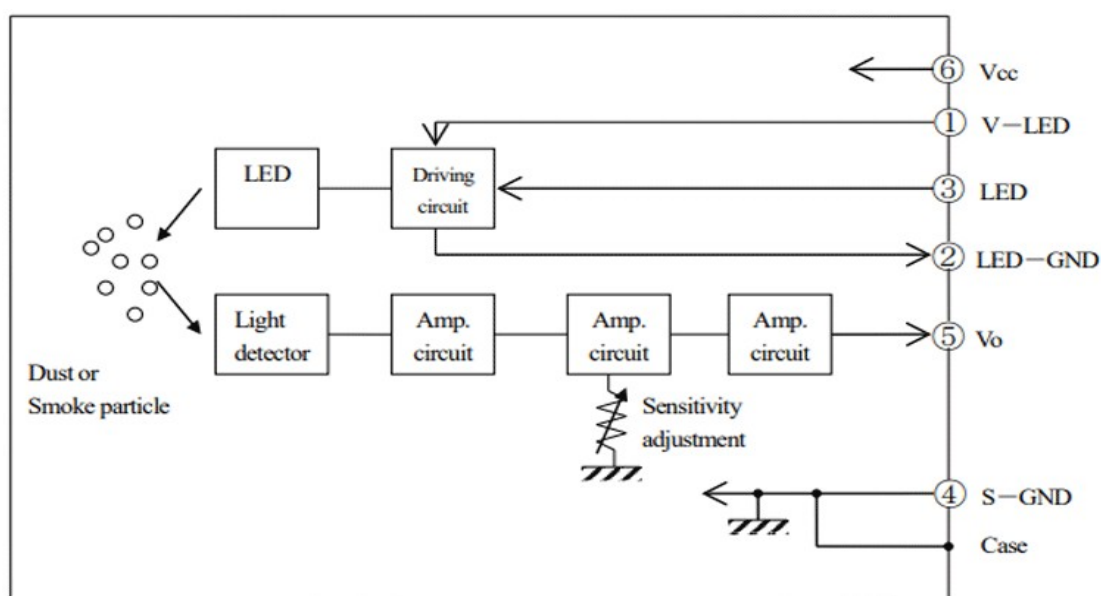
3.2. Sơ đồ kết nối với ESP32

- **Vout** : Xuất tín hiệu tương tự của cảm biến tới chân D34 của ESP32.
- **LED** : Nhận tín hiệu điều khiển từ chân D5 của ESP32 để vận hành đèn LED tích hợp.
- **S-GND, LED-GND** : Cả hai chân nối đất đều được kết nối với chân GND của ESP32.
- **VCC, V-LED** : Cả hai chân nguồn đều được kết nối với chân 3V3 của ESP32 để nhận nguồn.

Hình 3. Sơ đồ nối chân của GP2Y1010AU0F

3.3. Nguyên lý hoạt động

Nguyên lý hoạt động cơ bản của cảm biến này là quang học, cụ thể hơn là nguyên lý tán xạ ánh sáng. Không khí đi vào cảm biến thông qua một van nạp khí nơi đèn LED hồng ngoại bật tắt liên tục rơi vào không khí. Đèn hồng ngoại này chiếu sáng các hạt bụi có sẵn trong không khí. Đáp lại, một tín hiệu ánh sáng phân tán được tạo ra. Tín hiệu ánh sáng phân tán này được phát hiện bởi transistor quang của máy dò ánh sáng. Đầu ra của mạch tách sóng được khuếch đại bởi mạch khuếch đại nhiều tín hiệu. Sau đó, một tín hiệu ánh sáng khuếch đại được xử lý để có được nồng độ của các hạt bụi trong không khí. Cuối cùng, cảm biến bụi quang tạo ra tín hiệu điện áp tương tự trên chân Vo theo nồng độ của các hạt bụi hoặc khói trong không khí.



Hình 4. Nguyên lý hoạt động của cảm biến GP2Y1010AU0F

4. Cảm biến khí MQ-135

4.1. Giới thiệu

Cảm biến chất lượng không khí MQ135 được sử dụng với nhiều dòng cảm biến MQ, có thể tinh chỉnh được bằng biến trở. Cảm biến này có khả năng nhận diện nhiều loại khí như khí gas, khí CO, khí cacbonic,...

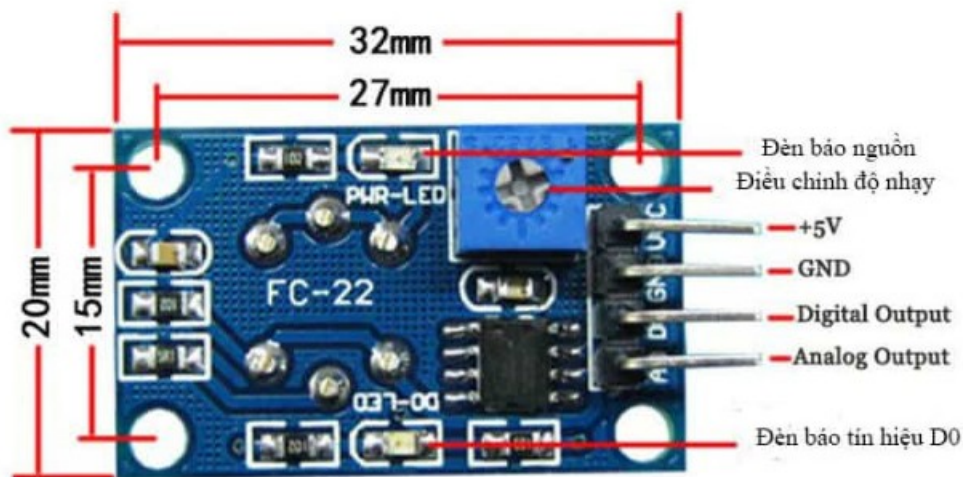
Dựa vào đó, ta có thể xây dựng nhiều chương trình nhận biết chất lượng không khí, từ đó gửi thông báo cho chúng ta nhằm mục đích bảo vệ sức khỏe.



- Điện áp nguồn: $\leq 24\text{VDC}$
- Điện áp của heater: $5\text{V} \pm 0.1 \text{ AC/DC}$
- Điện trở tải: thay đổi được ($2\text{k}\Omega$ - $47\text{k}\Omega$)
- Điện trở của heater: $33\Omega \pm 5\%$
- Công suất tiêu thụ của heater: ít hơn 800mW
- Khoảng phát hiện: 10 – 300 ppm NH₃, 10 – 1000 ppm Benzene, 10 – 300 Alcol

- Kích thước: 32mm*20mm

Hình 5. Cảm biến khí MQ-135

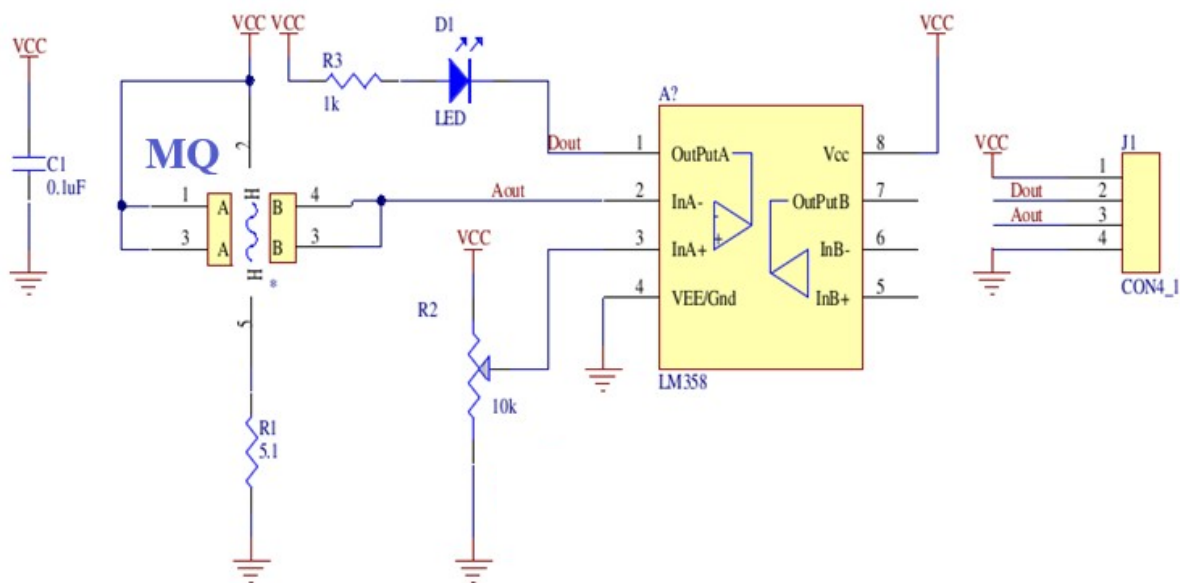


Hình 6. Sơ đồ mạch cảm biến khí MQ-135

4.2. Nguyên lý hoạt động

Cảm biến chất lượng không khí MQ135 hoạt động dựa trên nguyên lý phản ứng hóa học khi tiếp xúc các khí trong môi trường. Nguyên tắc hoạt động của nó là khi các khí trong môi trường như khí CO, khí LPG, khí methane, khí Hydro, khói... tiếp xúc với phần tử bên trong cảm biến, làm cho các electron được giải phóng vào Thiếc Dioxide cho phép dòng điện chạy qua cảm biến một cách tự do.

Khi được làm nóng, phần tử cảm biến sẽ tạo ra phản ứng hóa học với các khí tiếp xúc và làm thay đổi điện trở của phần tử cảm biến. Cảm biến MQ135 đo lường các biến đổi điện trở này và chuyển đổi chúng thành tín hiệu điện analog hoặc Digital.



Hình 7. Nguyên lý hoạt động cảm biến khí MQ135

4.2 Danh sách linh kiện sử dụng

Từ những linh kiện lựa chọn được sau khi so sánh, dưới đây là danh sách các linh kiện cần sử dụng để hoàn thiện sản phẩm “Đo lường chất lượng không khí”.

STT	Tên linh kiện	Số lượng
1	ESP32	1
2	Cảm biến nhiệt độ, độ ẩm DHT11	1
3	Cảm biến bụi GP2Y1010AU0F SHARP	1
4	Dây mạch và dây nguồn	Nhiều
5	Trở 5V	1

6	Tụ điện	1

Bảng 7. Danh sách các linh kiện sử dụng

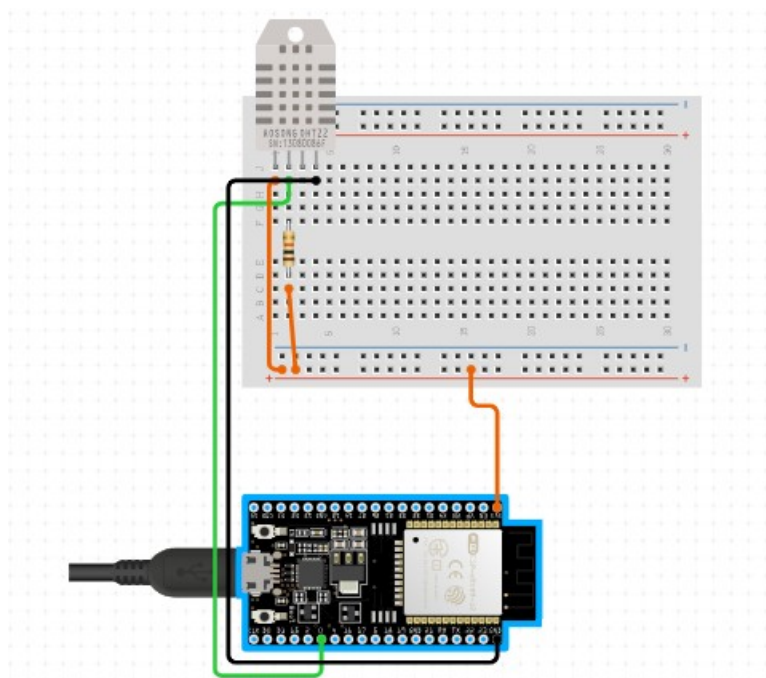
5. THIẾT KẾ HỆ THỐNG

5.1 Thiết kế phần cứng

5.1.1. Mạch sensor DHT-11

Đối với thiết kế phần cứng chúng tôi sẽ sử dụng vi xử lý ESP32 và cảm biến nhiệt độ, độ ẩm DHT11 để đánh giá chất lượng của hệ thống Cảm biến nhiệt độ và độ ẩm trước,

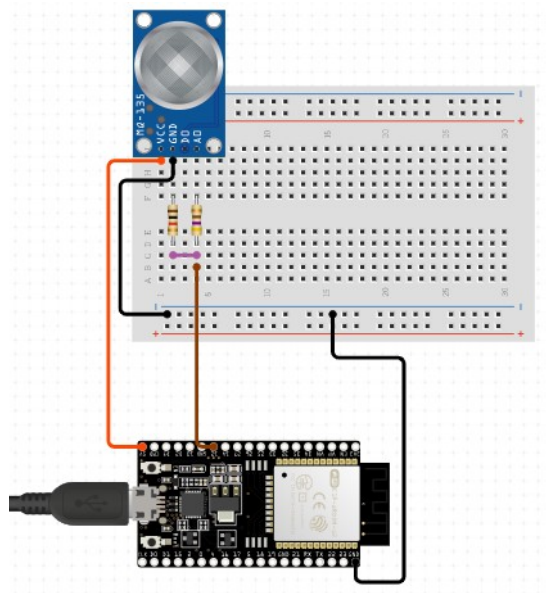
Cảm biến DHT-11 hoạt động dưới điện áp 3.3V – 5V có thể đo nhiệt độ từ 0 ° C đến 50 ° C và độ ẩm từ 20% đến 90% với độ chính xác ± 1 ° C và $\pm 1\%$.



Hình 8. Thiết kế phần cứng ESP32 và DHT-11

5.1.1. Mạch sensor MQ-135

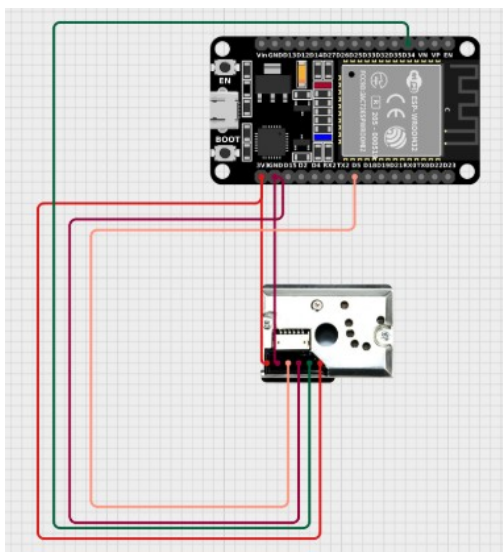
Cảm biến MQ-135 hoạt động dưới điện áp nguồn $\leq 24V$ DC, điện trở tải thay đổi được (2k Ω $\rightarrow$ 47k Ω), điện trở heater: 33 Ω , Với công suất hoạt động trong vùng 800mW và nồng độ phát hiện của một số chất: 10 - 300 ppm NH₃, 10 - 1000 ppm Benzen, 10 - 300 ppm Alcol.



Hình 9. Thiết kế phần cứng ESP32 và MQ-135

5.1.2. Sensor đo bụi mịn PM GP2Y1010AU0F

Điện áp hoạt động của cảm biến này trong khoảng từ 3.3 V đến 5V, tiêu thụ một lượng công suất trong khoảng 50mW.



nhận nguồn.

- **Vout** : Xuất tín hiệu tương tự của cảm biến tới chân D34 của ESP32.

- **LED** : Nhận tín hiệu điều khiển từ chân D5 của ESP32 để vận hành đèn LED tích hợp.

- **S-GND, LED-GND** : Cả hai chân nối đất đều được kết nối với chân GND của ESP32.

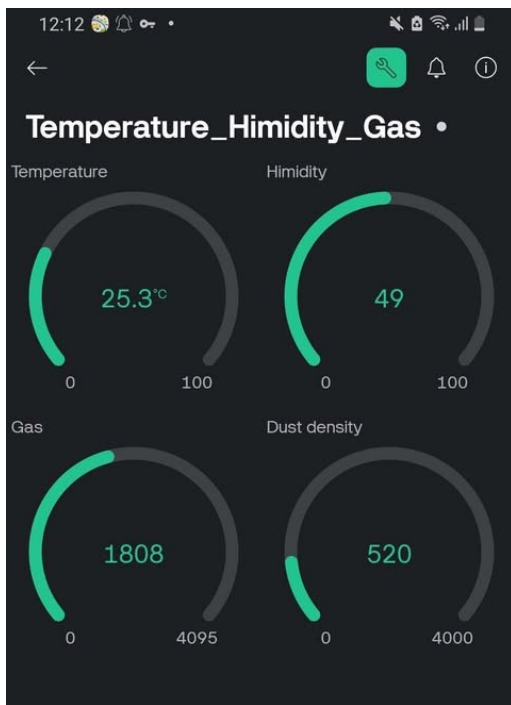
- **VCC, V-LED** : Cả hai chân nguồn đều được kết nối với chân 3V3 của ESP32 để

Hình 10. Thiết kế phần cứng ESP32 và PM GP2Y1010AU0F

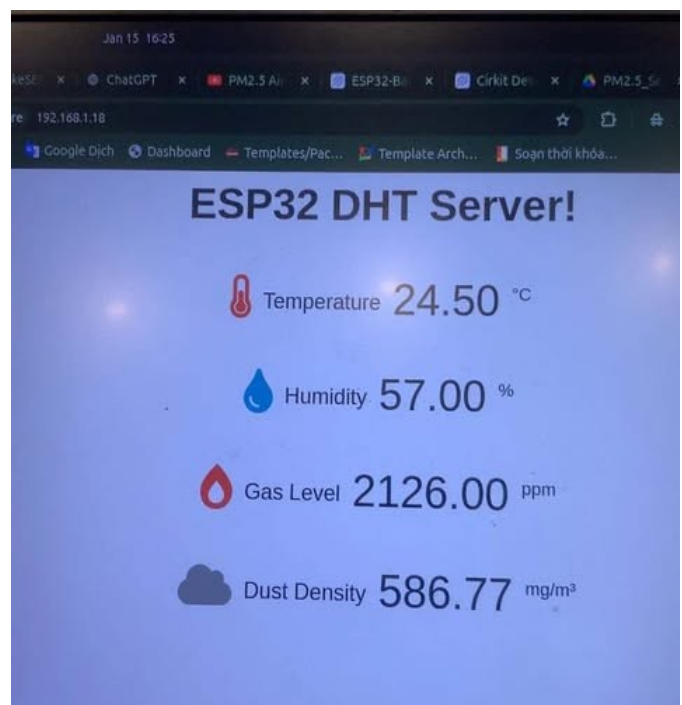
5.2. Thiết kế phần mềm

5.2.1. Phần mềm và thực hiện dự án :

- Thư viện : Arduino library for DHT11, DHT22, etc Temp&Humidity Sensors.
- Phần mềm : Visual StudioCode with platformio.
- Ngoài ra, phần mềm Blynk được sử dụng để giám sát chỉ số chất lượng không khí như nhiệt độ, độ ẩm, bụi, khí gas...



Hình 11. Giao diện phần mềm Blynk



Hình 12. Giao diện trên Web Server

5.2.2 Thư viện cho cảm biến

- Cảm biến nhiệt độ/độ ẩm DHT11: DHT sensor library for ESPx
- Cảm biến bụi (Sharp GP2Y1010AU0F: Giao tiếp Analog
- WiFi Library: Có sẵn trong core ESP32
- Thư viện cho Web Server: ESPAsyncWebServer - Để tạo web server hỗ trợ HTTP và WebSocket
- Thư viện hỗ trợ JSON: ArduinoJson - Để gửi dữ liệu cảm biến dạng JSON qua WebSocket hoặc HTTP

6. TRIỂN KHAI

6.1. Bảng phân chia công việc :

Nhóm đã phân công cụ thể công việc cho các thành viên như được đề cập trong Bảng 7

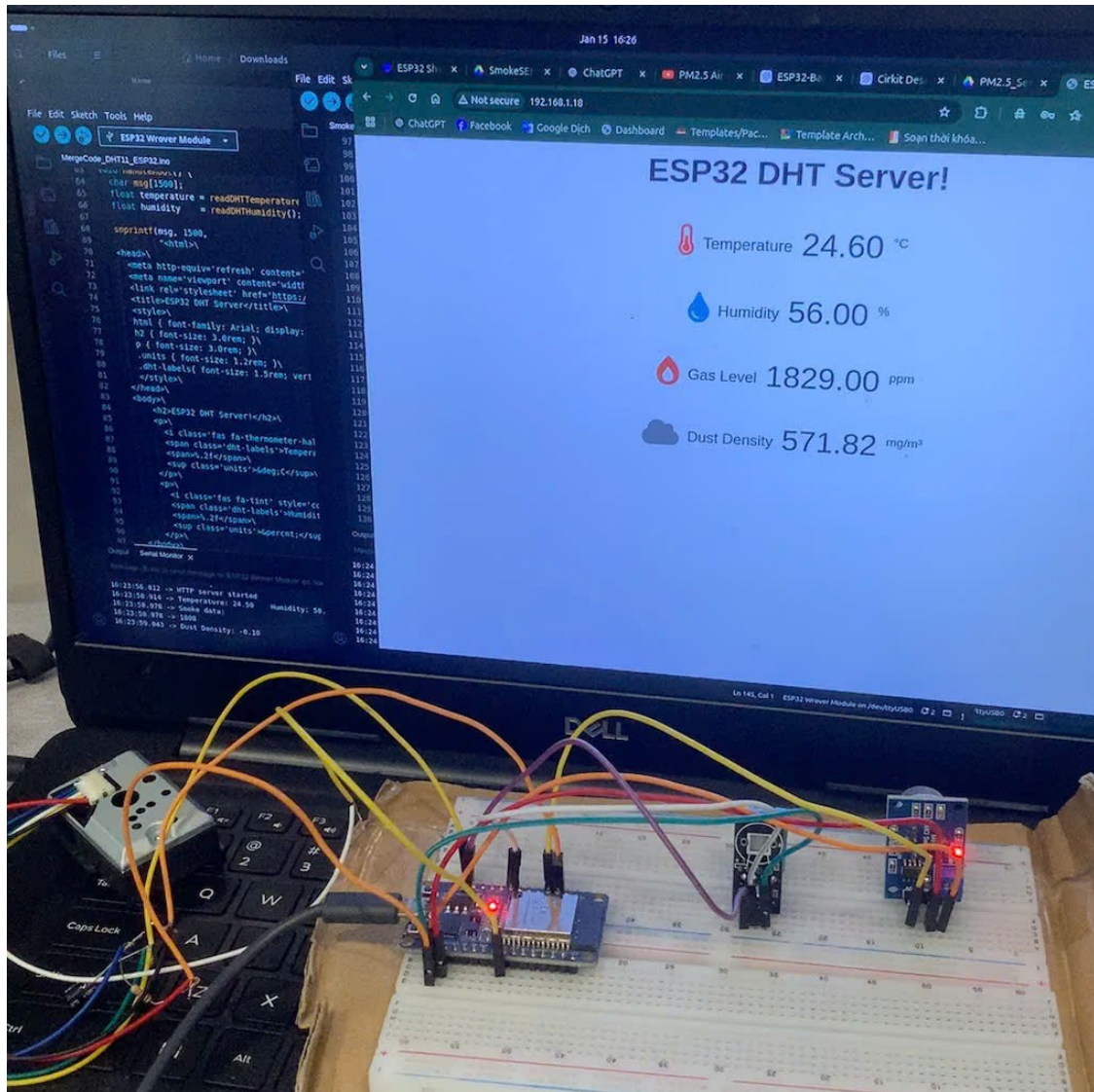
Bảng 7. Phân chia công việc theo các thành viên

Nhóm công việc	Công việc cụ thể	Nhân lực	Ghi chú
Phân tích đề tài	Tìm hiểu hiện trạng và vấn đề	cả nhóm	
	Định hướng sản phẩm, đưa ra mô hình giải quyết bài toán		
	Phân tích các yêu cầu		
Thiết kế phần cứng	Thiết kế sơ đồ khối	cả nhóm	Sau khi thiết kế từ tổng quan đến chi tiết cần đưa ra được cấu tạo của bộ mạch, các linh kiện sẽ sử dụng, datasheet của linh kiện, lắp ráp và kiểm thử lỗi và giá trị thực
	Thiết kế mạch và lắp mạch	Cả nhóm	
	Nghiên cứu các thiết bị hiện có và đề xuất chọn linh kiện	Đức + Bình	
	Nghiên cứu cấu tạo và phương thức truyền nối của các linh kiện	Cả nhóm	
	Thiết kế thư viện của linh kiện và layout	An	
Lập trình phần mềm	Thiết kế các sơ đồ logic	Bình	Lập trình bám sát vào các yêu cầu chức năng đã đặt ra
	Lập trình mã chạy và thực thi	Cả nhóm	
	Nghiên cứu phần ngoại vi Web server	Đức	
Hoàn thiện sản phẩm	Giám sát độ hoàn thiện	Bình	
	Tìm mua linh kiện	An	
	Thiết kế vỏ sản phẩm	Đức	
	Nạp code test hoàn thiện	cả nhóm	

6.2. Kết quả thực hiện

6.2.1. Kết quả mạch thực tế

Sau khi tiến hành mua linh kiện và lập trình trên vi xử lý, nhóm thực hiện hệ thống đo chất lượng không khí trên breadboard như hình :



Hình 13. Mạch thực tế sau khi hoàn thiện của nhóm

6.2.2. Kiểm thử

Trong quá trình thực hiện nhóm đã thực hiện kiểm thử hệ thống theo các kịch bản sau:

Bảng . Bảng các bài kiểm thử hệ thống

STT	Mục đích kiểm thử	Thủ tục kiểm thử	Kết quả mong muốn	Kết quả kiểm thử
1	Kiểm thử hiển thị trên App Blynk	1. Kết nối App Blynk với ESP32 2. Lập trình hiển thị các thông số nhiệt độ, độ ẩm, bụi, khí gas lên app	App hiển thị đầy đủ các thông số nhiệt độ, độ ẩm, bụi, khí gas	Đạt
2	Kiểm thử đầu ra của cảm biến DHT11	1. Kết nối cảm biến DHT11 với ESP32 2. Lập trình hiển thị các thông số nhiệt độ, độ ẩm, bụi, khí gas lên app 3. Thay đổi nhiệt độ, độ ẩm, bụi, khí gas xung quanh cảm biến bằng các nguồn nhiệt, bụi, gas	Các thông số nhiệt độ, độ ẩm, bụi, khí gas hiển thị và thay đổi theo sự thay đổi của các nguồn nhiệt, bụi, khí gas	Đạt
3	Kiểm thử chức năng hiển thị trên Web	1. Đăng nhập vào mạng WiFi trên ESP32 2. Truy cập vào địa chỉ IP của trang Web	Các thông số hiển thị trên trang Web giống với các thông số trên App Blynk	Đạt

6.2.3. Đánh giá

Hệ thống hoạt động ổn định, các kết quả kiểm thử cho thấy các thông số nhiệt độ, độ ẩm, bụi, và khí gas được hiển thị chính xác trên cả ứng dụng App Blynk và giao diện Web.

Tốc độ cập nhật dữ liệu giữa cảm biến và các nền tảng hiển thị là nhanh chóng, không có độ trễ đáng kể, đảm bảo phản ánh đúng các thay đổi của môi trường thực tế.

Kết nối WiFi và giao tiếp giữa ESP32 với App Blynk cũng như trang Web đạt yêu cầu, duy trì ổn định trong suốt quá trình thử nghiệm.

Không phát hiện lỗi phần cứng hay sự cố về phần mềm trong quá trình chạy thử, các chức năng được thực hiện theo đúng yêu cầu thiết kế.

Đáp ứng tốt yêu cầu giám sát và hiển thị các thông số môi trường theo thời gian thực, có khả năng triển khai trong thực tế.

Có thể phát triển thêm các tính năng mở rộng như cảnh báo khi thông số vượt ngưỡng an toàn hoặc tích hợp điều khiển từ xa thông qua App hoặc Web.

6.2.4. Code

```
#include <WiFi.h>
#include <WiFiClient.h>
#include <WebServer.h>
#include <ESPmDNS.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>

const char* ssid = "HaiBinh";
const char* password = "0888816858";

const int smokeA0 = 35;
int sensorThres = 100;
int smokeData;

#define DHTPIN 26
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

const int measurePin = 34; // Dust sensor pin
const int ledPower = 5; // LED driver pin for dust sensor

int samplingTime = 280; // Sampling time in microseconds
int deltaTime = 40; // Delay time between measurements
int sleepTime = 9680; // Sleep time in microseconds

float voMeasured = 0; // Measured voltage from dust sensor
float calcVoltage = 0; // Calculated voltage
float dustDensity = 0; // Calculated dust density

BlynkTimer timer;
WebServer server(80);

float readDHTTemperature();
float readDHTHumidity();
```

```

void handleRoot();
void sendSensor();
void sendSmokeSensor();
void sendDustSensor();

void setup() {
  Serial.begin(115200);

  // Initialize DHT sensor
  dht.begin();

  // Initialize Blynk
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, password);

  // Wait for WiFi connection
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected to: ");
  Serial.println(ssid);
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());

  // Initialize MDNS
  if (MDNS.begin("esp32")) {
    Serial.println("MDNS responder started");
  }

  // Initialize WebServer
  server.on("/", handleRoot);
  server.begin();
  Serial.println("HTTP server started");

  // Set pin modes
  pinMode(smokeA0, INPUT);
  pinMode(ledPower, OUTPUT);

```



```

// Set timers
timer.setInterval(2000L, sendSensor);
timer.setInterval(2000L, sendSmokeSensor);
timer.setInterval(2000L, sendDustSensor);
}

void loop() {
  Blynk.run();
  timer.run();
  server.handleClient();
  delay(1000);
}

void handleRoot() {
  char msg[2000];
  float temperature = readDHTTemperature();
  float humidity = readDHTHumidity();
  float gasData = analogRead(smokeA0);
  float dustData = dustDensity;

  snprintf(msg, 2000,
    "<html>\n
    <head>\n
    <meta http-equiv='refresh' content='4'/>\n
    <meta name='viewport' content='width=device-width, initial-
    scale=1'>\n
    <link                                rel='stylesheet'
    href='https://use.fontawesome.com/releases/v5.7.2/css/all.css'
    integrity='sha384-
    fnmOCqbTIWIlj8LyTjo7mOUStjsKC4pOpQbqyi7RrhN7udi9RwhKkMHpvL
    bHG9Sr' crossorigin='anonymous'>\n
    <title>ESP32 DHT Server</title>\n
    <style>\n
    html { font-family: Arial; display: inline-block; margin: 0px auto; text-
    align: center; }\n
    h2 { font-size: 3.0rem; }\n
    p { font-size: 3.0rem; }\n
    .units { font-size: 1.2rem; }\n
    .dht-labels{ font-size: 1.5rem; vertical-align:middle; padding-bottom:
    15px;}\n
  ");
  server.send(200, "text/html", msg);
}

```

```

</style>\
</head>\
<body>\
<h2>ESP32 DHT Server!</h2>\
<p>\
<i class='fas fa-thermometer-half' style='color:#ca3517;'></i>\
<span class='dht-labels'>Temperature</span>\
<span>%.2f</span>\
<sup class='units'>&deg;C</sup>\
</p>\
<p>\
<i class='fas fa-tint' style='color:#00add6;'></i>\
<span class='dht-labels'>Humidity</span>\
<span>%.2f</span>\
<sup class='units'>&percent;</sup>\
</p>\
<p>\
<i class='fas fa-burn' style='color:#ca3517;'></i>\
<span class='dht-labels'>Gas Level</span>\
<span>%.2f</span>\
<sup class='units'>ppm</sup>\
</p>\
<p>\
<i class='fas fa-cloud' style='color:#7e7e7e;'></i>\
<span class='dht-labels'>Dust Density</span>\
<span>%.2f</span>\
<sup class='units'>mg/m&sup3;</sup>\
</p>\
</body>\
</html>",

```

```

temperature, humidity, gasData, dustData
);
server.send(200, "text/html", msg);
}

```

```

void sendSensor() {
float t = readDHTTemperature();
float h = readDHTHumidity();

```

```

if (t == -1 || h == -1) {
  Serial.println("Failed to read from DHT sensor!");
  return;
}

Blynk.virtualWrite(V0, t);
Blynk.virtualWrite(V1, h);
Serial.print("Temperature: ");
Serial.print(t);
Serial.print(" Humidity: ");
Serial.println(h);
}

void sendSmokeSensor() {
  smokeData = analogRead(smokeA0);
  Blynk.virtualWrite(V2, smokeData);

  Serial.println("Smoke data: ");
  Serial.println(smokeData);

  if (smokeData > 2000) {
    Serial.println("Send event noti");
    Blynk.logEvent("gaz_detected", "Gas detected");
  }
}

void sendDustSensor() {
  digitalWrite(ledPower, LOW); // Power on the LED
  delayMicroseconds(samplingTime); // Wait for the sampling time

  voMeasured = analogRead(measurePin); // Read the dust value from
the sensor

  delayMicroseconds(deltaTime); // Delay for delta time
  digitalWrite(ledPower, HIGH); // Turn the LED off
  delayMicroseconds(sleepTime); // Sleep for the remaining time

  calcVoltage = voMeasured * (5.0 / 1024.0); // Convert analog value to
voltage (0-5V mapped to 0-1023)

```

```
dustDensity = 170 * calcVoltage - 0.1; // Calculate the dust density in
mg/m³

Blynk.virtualWrite(V3, dustDensity);
Serial.print("Dust Density: ");
Serial.println(dustDensity);
}

float readDHTTemperature() {
float t = dht.readTemperature();
if (isnan(t)) {
Serial.println("Failed to read temperature!");
return -1;
} else {
return t;
}
}

float readDHTHumidity() {
float h = dht.readHumidity();
if (isnan(h)) {
Serial.println("Failed to read humidity!");
return -1;
} else {
return h;
}
}
```

----- The End -----